DEEP REINFORCEMENT LEARNING 365.382 (SS26)

Model-based RL: Learning Models

Sohvi Luukkonen
LIT AI LAB
Institute for Machine Learning

JKU
JOHANNES KEPLER UNIVERSITY LINZ

Model-based RL: Thinking Ahead

- Humans and animals simulate future outcomes before acting
- Chess players explore many candidate moves mentally
- Robots can plan trajectories without touching obstacles

Key question: can an RL agent do the same?

- Build or use a model of the environment
- Use it to plan without collecting real data
- Potentially massive gains in sample efficiency

Planning Given a Model

- Dynamic programming: full-MDP Bellman sweeps
- Forward search and MCTS: tree-structured lookahead
- AlphaGo / AlphaZero: MCTS guided by neural networks
- Trajectory optimisation: CEM for continuous control

All of that assumed we had access to:

$$p(s' \mid s, a), \qquad r(s, a)$$

In most real problems the model is not given — it must be learned.

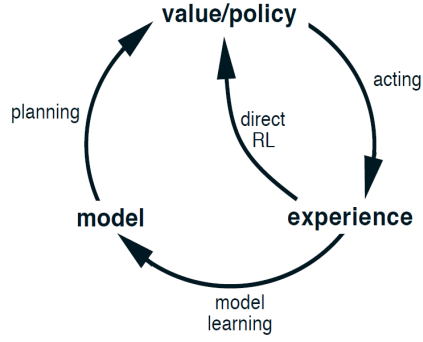
The Model-Based RL Loop

Model-free RL:

- Interact $\rightarrow$ observe (s, a, r, s')
- Update policy / value directly
- No model of the world

Model-based RL:

- Interact $\rightarrow$ collect transitions
- Fit a model $\hat{p}_\phi(s' \mid s, a)$, $\hat{r}_\phi(s, a)$
- Plan / imagine with the model
- Update policy from real + imagined data



Lecture Outline

Learning a Model from Transitions

What Does the Model Need to Predict?

Minimal model for one-step rollouts:

Transition model: predicts next state – tells us where we go

$$s_{t+1} \approx f_\phi(s_t, a_t) \qquad \text{(deterministic)}$$

$$s_{t+1} \sim p_\phi(\cdot \mid s_t, a_t) \qquad \text{(stochastic)}$$

Reward model: predicts immediate reward – tells us what we get

$$\hat{r}_t = r_\phi(s_t, a_t)$$

Together they define a simulated MDP: given (s, a) produce (s', r) without touching the real environment.

Supervised Learning for the Model

Given dataset $\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1})\}$ collected from the environment:

Transition model (maximum likelihood / regression):

$$\mathcal{L}_{\text{trans}}(\phi) = -\frac{1}{|\mathcal{D}|} \sum_{(s,a,s') \in \mathcal{D}} \log p_\phi(s' \mid s, a) \quad \text{or} \quad \frac{1}{|\mathcal{D}|} \sum \left\| s' - f_\phi(s, a) \right\|^2$$

Reward model (regression):

$$\mathcal{L}_{\text{rew}}(\phi) = \frac{1}{|\mathcal{D}|} \sum_{(s,a,r) \in \mathcal{D}} \big(r - r_\phi(s, a) \big)^2$$

- Architecture: MLP for low-dim states, CNN for images, RNN for partial observability
- Reward model is often simpler (scalar output); sometimes reward is known analytically

Data Collection Strategy

The quality of the learned model depends critically on which data is collected.

- Random policy: broad state coverage, safe, but many irrelevant regions
- Current policy: data matches where the agent actually operates $\rightarrow$ model is more accurate where it matters
- Mixture / replay buffer: collect with evolving policy, keep all past transitions
- Active / uncertainty-driven: query the environment where the model is uncertain

Why Learned Models Fail

Problem 1: Distributional Shift

The model is trained on transitions from the data-collection policy.

When the planning policy is different, it visits states the model has never seen:

$$(s, a) \sim \pi_{\text{plan}} \neq (s, a) \sim \pi_{\text{data}}$$

- Model errors are undefined outside the training distribution
- The planner may confidently exploit model artefacts
- Similar to covariate shift in imitation learning (recall DAgger)

Mitigation:

- Re-collect data under the planning policy (iterative / online MBRL)
- Uncertainty-aware planning: avoid low-confidence regions

Problem 2: Compounding Errors

Even a model with small per-step error can diverge over long horizons.

Suppose per-step error ϵ; after H steps:

$$\text{total error} \leqslant H \cdot \epsilon \qquad \text{(linear in horizon)}$$

(can be exponential for divergent dynamics)

- Small $\epsilon \approx 0.01$, horizon $H = 100 \Rightarrow$ error ≈ 1.0 (order of state scale)
- The model generates a trajectory far from any real trajectory
- Value estimates computed on these trajectories are meaningless

Mitigation: use short rollouts, combine with real data, penalise uncertainty.

Problem 3: Model Exploitation

A policy optimised against a learned model will find its weaknesses.

- The model is an imperfect proxy for the real environment
- Policy gradient or planning will over-optimise the model
- Actions that look great in simulation may be catastrophic in reality

Classic example: a robot that discovers a model flaw and vibrates indefinitely to accumulate infinite simulated reward.

Mitigations:

- Limit planning horizon H (reduce opportunity for exploitation)
- Penalise uncertainty: pessimistic / conservative planning
- Frequently re-train the model on new real data

The Accuracy–Usability Trade-off

Short horizon (H small)

- Low compounding error
- More accurate predictions
- Limited planning depth
- Need good value function at leaf

Long horizon (H large)

- Richer signal for policy
- High compounding error
- Model exploitation likely
- Requires very accurate model

Practical recommendation:

keep H short (5–20 steps) and combine imagined data with real data.

Re-planning at Every Step

Model Predictive Control (MPC)

MPC is a classical control strategy that plans over a short horizon and re-plans at every step.

Receding-horizon loop:

1. Given current state s_t, optimise action sequence $a_t, \ldots, a_{t+H-1}$ under the learned model:
$$\mathbf{a}^* = \arg\max_{a_t,\ldots,a_{t+H-1}} \sum_{k=0}^{H-1} \gamma^k \hat{r}(s_{t+k}, a_{t+k})$$
2. Execute only a_t^*; discard the rest
3. Observe s_{t+1}; repeat

- Re-planning at each step corrects for model errors
- Computationally expensive: inner optimisation every step
- Works with any black-box model

Cross-Entropy Method (CEM) for Planning

CEM is a simple, derivative-free optimiser used inside MPC.

Represent the distribution over each action a_t as an independent Gaussian:

$$p(a_t) = \mathcal{N}(\mu_t,\, \sigma_t^2 I), \qquad t = 0, \ldots, H-1$$

Algorithm (one MPC step):

1. Initialise $\mu_t = \mathbf{0}$, $\sigma_t = \sigma_0$ for all t
2. Sample K action sequences: $\mathbf{a}^{(i)} \sim \prod_t \mathcal{N}(\mu_t, \sigma_t^2 I)$
3. Simulate each sequence under $\hat{p}_\phi$; compute total return $R^{(i)}$
4. Keep top-N elite sequences (highest $R^{(i)}$)
5. Refit: $\mu_t \leftarrow$ mean of elites at step t; $\sigma_t^2 \leftarrow$ var of elites at step t
6. Repeat 2–5 for several iterations; execute μ_0

CEM: Key Properties

- Separate distribution per time step: $\mathcal{N}(\mu_t, \sigma_t^2)$ for each $t \rightarrow$ trajectories are not constrained to be smooth or physically consistent
- Derivative-free: works with any black-box model (even non-differentiable)
- Parallelisable: K trajectory evaluations are independent
- Simple to implement: no gradient computation through the model

Limitations:

- Scales poorly with horizon H and action dimension d_a
- Purely exploits within the current search distribution (local optima)
- No memory between MPC steps (warm-starting helps)

Dyna: Learning from Imagined Transitions

Dyna: The Core Idea

Dyna is an early and influential framework for combining model learning with model-free RL.

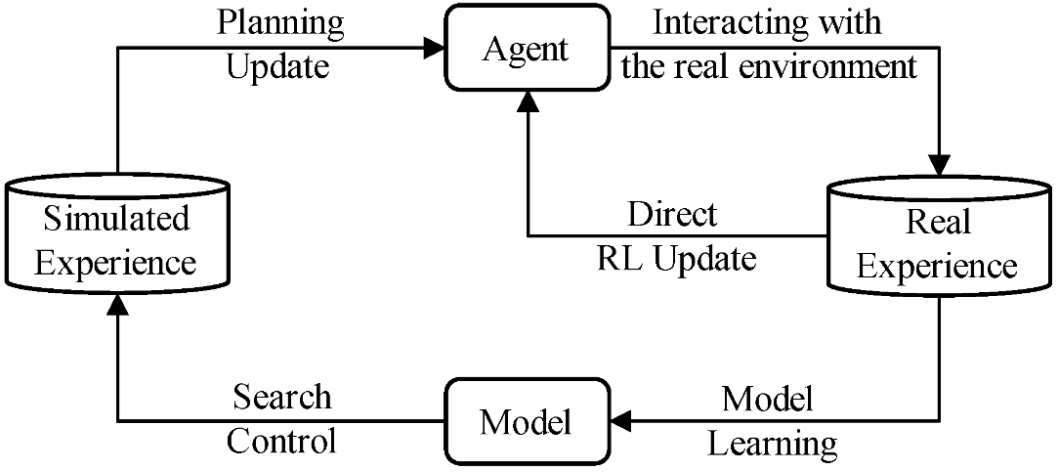
Key insight: imagined transitions from a learned model can be treated like ordinary experience.

Real experience: update the model and the value function / policy.
Imagined experience: update the value function / policy only.

- Each real transition can be followed by K model-generated updates
- A good model can greatly improve sample efficiency
- The model does not have to be used for explicit lookahead planning

Dyna turns a learned model into an additional source of training data.

Dyna Algorithm



Dyna-Q Algorithm

Inputs: Q-function Q, learned model $\hat{p}_\phi$, replay buffer $\mathcal{D}$

1. **Act:** choose $a_t \leftarrow \varepsilon$-greedy$(Q, s_t)$; observe r_t, s_{t+1}
2. **Learn from real experience:**

$$(s_t, a_t, r_t, s_{t+1})$$

   - ☐ update Q
   - ☐ update the model $\hat{p}_\phi$
   - ☐ store the transition in $\mathcal{D}$

3. **Learn from imagined experience:** repeat K times:

$$(s, a) \sim \mathcal{D}, \qquad (r', s') \sim \hat{p}_\phi(\cdot \mid s, a)$$

   update Q with the imagined transition (s, a, r', s')

4. Advance to s_{t+1} and repeat

Dyna-Q: Effect of Imagined Steps

- $K = 0$: ordinary model-free Q-learning
- $K > 0$: each real step is supplemented by K model-generated updates

Benefit: with an accurate model, many more value updates can be made per real environment interaction.

Caveats:

- A wrong model produces misleading imagined transitions
- In non-stationary environments, the model can become outdated
- Standard Dyna samples previously visited (s, a) pairs and therefore has limited exploration

Dyna is the blueprint for modern MBRL:
real data $\rightarrow$ learned model; learned model $\rightarrow$ synthetic data $\rightarrow$ policy.

Dyna-Q+: Using the Model for Exploration

Standard Dyna improves learning from already observed experience.

Dyna-Q+ adds an exploration bonus for state-action pairs that have not been tried for a long time:

$$r^+(s,a) = r(s,a) + \kappa\sqrt{\tau(s,a)}$$

- $\tau(s,a)$: time since action a was last tried in state s
- κ: strength of the exploration bonus

Dyna-Q: use the model to learn faster.
Dyna-Q+: also use the model to explore stale or neglected actions.

This is especially useful when the environment may have changed, or when the agent needs a reason to revisit old choices.

Model-Based Policy Optimization

MBPO: Motivation

Model-Based Policy Optimization (MBPO) combines a learned dynamics model with an off-policy actor-critic.

Problem: long rollouts from an imperfect model quickly drift away from realistic trajectories.

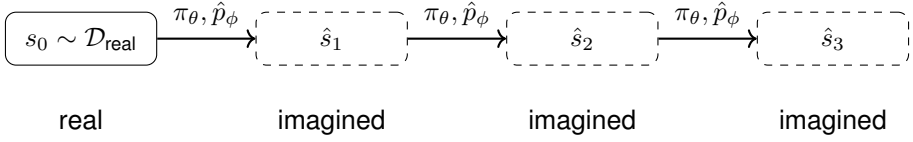
MBPO addresses this with two design choices:

1. Short model rollouts starting from real states in the replay buffer
2. Mixed replay: train SAC on a mixture of real and imagined transitions

Start from states the model has seen, imagine only a few steps, and use the synthetic transitions to improve the policy.

Janner et al., NeurIPS 2019

MBPO: Short Rollouts from Real States



- The first state comes from the real replay buffer
- The model is used only for a short rollout of length k
- The generated transitions are added to a model buffer

Short rollouts reduce compounding error while still producing useful extra data.

MBPO Algorithm

Inputs: policy π_θ, model ensemble $\{\hat{p}_{\phi_i}\}$, real buffer $\mathcal{D}_{\text{real}}$, model buffer $\mathcal{D}_{\text{model}}$

1. **Collect real data:** roll out π_θ in the real environment and add transitions to $\mathcal{D}_{\text{real}}$
2. **Train the model:** fit the ensemble on transitions from $\mathcal{D}_{\text{real}}$
3. **Generate imagined data:** sample $s_0 \sim \mathcal{D}_{\text{real}}$ and roll out the current policy for k model steps
4. **Store imagined transitions:** add the generated transitions to $\mathcal{D}_{\text{model}}$
5. **Update the policy:** train SAC on minibatches from $\mathcal{D}_{\text{real}} \cup \mathcal{D}_{\text{model}}$
6. Repeat

Key hyperparameter: rollout length k, typically short, e.g. 1–5 steps in MuJoCo tasks.

Why Short Rollouts?

MBPO balances two competing effects.

Rollout length	Advantage	Risk
$k = 0$	no model bias	no synthetic data
small k	useful extra data	limited model error
large k	more imagined experience	compounding error

The goal is not to build a perfect simulator.
The goal is to use the model only where it is accurate enough.

This is why MBPO usually uses very short rollouts and frequently retrains the model on new real data.

MBPO: Results and Limitations

Empirical result:

- Matches strong model-free SAC performance with far fewer real environment steps on MuJoCo locomotion tasks
- Short rollouts, sometimes even $k = 1$, already work well
- The model ensemble is important for reducing model bias and estimating uncertainty

Limitations:

- Works best for smooth, continuous-control dynamics
- Does not directly solve high-dimensional image-based control
- Still depends on accurate reward and termination predictions
- Synthetic data can hurt if the model is wrong or rollouts are too long

Dyna vs. MBPO

MBPO follows the same Dyna principle, but adds safeguards for deep RL.

Aspect	Dyna-Q	MBPO
Learner	Q-learning	SAC
Model	simple / tabular	neural ensemble
Imagined data	one-step samples	short rollouts
Start states	replayed states or pairs	real replay states
Error control	limited	short k, ensemble, mixed replay

MBPO is best seen as a modern Dyna-style algorithm designed to control model error.

Probabilistic Models and Ensembles

Why Uncertainty Matters

A deterministic model gives a single prediction:

$$s' = f_\phi(s, a)$$

This can be dangerous in model-based RL.

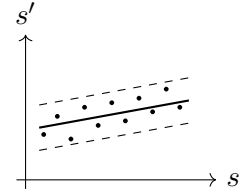
- The planner may trust predictions in regions with little data
- Small model errors can compound over long rollouts
- The policy may exploit confident but wrong predictions

A useful model should not only predict what may happen,
but also how reliable the prediction is.

Two Sources of Uncertainty

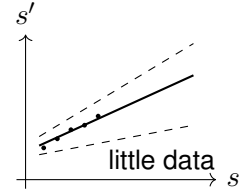
Aleatoric uncertainty: inherent randomness in the environment.

- Example: stochastic transitions, noisy sensors, random bounces
- Cannot be removed by collecting more data



Epistemic uncertainty: uncertainty due to limited knowledge.

- Example: the model is queried far from the training data
- Can be reduced by collecting more relevant data



Aleatoric uncertainty is part of the environment.
Epistemic uncertainty is part of the model.

Gaussian Neural Network Models

A probabilistic neural network outputs a mean and variance:

$$p_\phi(s' \mid s, a) = \mathcal{N}(\mu_\phi(s, a), \Sigma_\phi(s, a))$$

Training maximises the likelihood of observed transitions:

$$\mathcal{L}(\phi) = \sum_{(s,a,s') \in \mathcal{D}} \log \mathcal{N}\big(s'; \mu_\phi(s, a), \Sigma_\phi(s, a)\big)$$

- The mean predicts the most likely next state
- The variance captures noisy or ambiguous outcomes
- A diagonal covariance is often used for efficiency

What Does a Gaussian Model Capture?

A single Gaussian neural network can model aleatoric uncertainty.

- Large $\Sigma_\phi(s, a)$ means the transition is predicted to be noisy or hard to predict
- This is useful when the environment itself is stochastic
- But it does not reliably tell us whether the model is outside its training data

Predictive variance is not the same as knowing what the model does not know.

To estimate epistemic uncertainty, we need model disagreement.

Deep Ensembles for Epistemic Uncertainty

Train M independent models:

$$\{p_{\phi_i}(s' \mid s, a)\}_{i=1}^{M}$$

Each model is trained on the same data, but with different random seeds, initialisations, or bootstrapped data.

Ensemble prediction:

$$p_{\text{ens}}(s' \mid s, a) = \frac{1}{M} \sum_{i=1}^{M} p_{\phi_i}(s' \mid s, a)$$

- If all models agree, the prediction is more reliable
- If models disagree, the query may be out-of-distribution
- This disagreement approximates epistemic uncertainty

Uncertainty Decomposition in Ensembles

For Gaussian ensemble members, the total predictive variance can be decomposed:

$$\mathrm{Var}_{\mathrm{ens}}[s'] = \underbrace{\frac{1}{M}\sum_i \Sigma_{\phi_i}}_{\text{within-model variance}} + \underbrace{\frac{1}{M}\sum_i (\mu_{\phi_i} - \bar{\mu})^2}_{\text{between-model disagreement}}$$

Term	Interpretation	Uncertainty type
Within-model variance	noisy transitions	aleatoric
Between-model disagreement	lack of data / OOD query	epistemic

In MBRL, epistemic uncertainty is especially important: it tells the planner where the model should not be trusted.

How Ensembles Help in Model-Based RL

Ensembles can be used in several ways.

- Model selection: sample one model for each imagined rollout
- Uncertainty-aware planning: avoid actions that lead to high model disagreement
- Pessimistic value estimates: penalise rewards or values in uncertain regions
- Active data collection: collect real transitions where the model is uncertain

Ensembles help prevent the agent from exploiting model errors.

PETS: Planning with an Ensemble

Probabilistic Ensembles with Trajectory Sampling (PETS) is a model-based RL method for continuous control.

Component	Role	Purpose
Probabilistic ensemble	dynamics model	uncertainty
CEM	action-sequence search	planning
Trajectory sampling	ensemble rollouts	uncertainty propagation

PETS uses model uncertainty while evaluating candidate action sequences.

Chua et al., NeurIPS 2018

PETS: Trajectory Sampling

During CEM planning, each candidate action sequence is evaluated by rolling out several possible future trajectories, called particles, through the ensemble.

TS1: one model per rollout

$$\hat{p}_{\phi_2} \to \hat{p}_{\phi_2} \to \hat{p}_{\phi_2} \to \cdots$$

One coherent model hypothesis for the whole imagined trajectory.

TS∞: new model each step

$$\hat{p}_{\phi_2} \to \hat{p}_{\phi_5} \to \hat{p}_{\phi_1} \to \cdots$$

Each transition is sampled from the ensemble mixture.

Trajectory sampling makes disagreement between models affect planning.

PETS: Algorithm

At each real environment step:

1. Initialise CEM distribution over H-step action sequences
2. Repeat for K CEM iterations:
   - ☐ Sample N candidate sequences $A^{(j)}$
   - ☐ Score each via ensemble rollouts
   - ☐ Keep top-E elite sequences
   - ☐ Refit distribution to elites
3. Execute first action of the best sequence
4. Observe real s'; add to dataset; update ensemble

Scoring a candidate sequence A:

- Spawn P particles, all starting at s_t
- Each particle steps through A by:
  - ☐ Sampling a random ensemble member at each step
  - ☐ Stepping forward: $(s', r) \sim \hat{p}_{\phi_i}(\cdot \mid s, a)$
- Particle return: sum of predicted rewards along its trajectory
- Score of A: average return over all P particles

PETS = MPC with CEM, where candidate sequences are scored by rolling out particles through the probabilistic ensemble.

Latent World Models

From Transition Models to World Models

Earlier, our model was mostly a one-step transition model:

$$(s_{t+1}, r_t) \sim p_\phi(s_{t+1}, r_t \mid s_t, a_t)$$

This assumes that the state s_t is already a useful representation for prediction and control.

World-model view: learn an internal state representation and predict future experience from it:

$$z_t = e_\phi(o_{\leqslant t}, a_{<t}), \qquad z_{t+1}, r_t \sim p_\phi(z_{t+1}, r_t \mid z_t, a_t)$$

A world model is not just a next-state predictor.
It learns what state information the agent should carry forward for imagination and decision making.

Why Learn a Latent State?

The raw observation o_t is often not the right state for control.

- It can be high-dimensional: images, sensors, text, multimodal inputs
- It can be partial: one observation may not reveal the full state
- It can contain irrelevant variation: background, lighting, noise
- It may miss history-dependent information: velocity, memory, hidden variables

A latent state z_t can summarize the useful information from the history:

$$z_t = e_\phi(o_{\leqslant t}, a_{<t})$$

The goal is not perfect reconstruction.
The goal is a state representation useful for control.

Examples of Latent World-Model Agents

- World Models: learn latent dynamics from observations and optimise a controller in imagined rollouts.
- Dreamer: learn recurrent latent dynamics and train an actor-critic in imagined rollouts.
- MuZero: learn value-equivalent latent dynamics and run MCTS in latent space.

Same high-level idea: learn an internal model, then use imagined futures to improve decisions.

Ha & Schmidhuber, 2018; Hafner et al., 2020; Schrittwieser et al., 2020

Summary: When Model-Based RL Helps

Summary: What We Covered Today

1. Learning a model from transitions: supervised regression / MLE
2. Why models fail: distributional shift, compounding errors, exploitation
3. MPC + CEM: receding-horizon planning with a learned model
4. Dyna: interleave real and imagined experience for any model-free algorithm
5. MBPO: short rollouts from real states + SAC; $\sim 20\times$ more efficient
6. Ensembles: separate aleatoric / epistemic uncertainty; used in PETS
7. Latent world models: plan in compact learned state space (World Models, Dreamer, MuZero)

When Does Model-Based RL Help?

MBRL tends to work best when:

- Real data is expensive: robotics, chemistry, clinical trials
- Dynamics are smooth and learnable: continuous locomotion, manipulation
- Short planning horizons suffice: model errors stay bounded
- Reward is dense: easy to fit a reward model

MBRL is harder when:

- Observations are high-dimensional and complex (raw pixels — needs latent model)
- Dynamics are discontinuous or stochastic (contacts, multi-agent)
- Reward is sparse (hard to signal planning quality)
- Distribution shift is severe (rapidly changing environment)

Looking Ahead

Open challenges in model-based RL:

- Scaling latent world models to open-ended, long-horizon tasks
- Combining MBRL with large language / foundation models
- Certified safety using model-based constraints
- Transfer of learned models across tasks